A Self-Sustaining Ecosystem for Mesa Examples: Automating Quality, Maintenance, and Discoverability

GSoC 2026 Project Proposal

Contributor: Nandhakumar S

GitHub: @Nandha-kumar-S

Timezone: IST (UTC+5:30)

Proof Of Work & Technical Prototype: To prove the viability of this proposed architecture, I have already built and tested a working prototype of the Matrix CI, the PR Triage Bot, and the Metadata UI Gallery Builder.

You can view the sandbox here:
https://github.com/Nandha-kumar-S/GSoC-learning-space

1. Project Overview & Motivation....2
1.1. Target Audience (User Personas)....3
1.2. Scope & Philosophy....3
2. Technical Architecture....4
2.1. Pillar 1: Automated Developer Experience (DevEx) & Standardization....4
2.2. Pillar 2: Strict Isolation & Dynamic Matrix CI (The Infrastructure)....8
2.3. Pillar 3: Sustainable Maintenance & Decentralized Ownership (The Autopilot)....13
2.4. Pillar 4: Metadata-Driven Discoverability (The Gallery UI)....17
2.4.1. Anatomy of the Proposed UI (Feature Breakdown)....19
Supporting Infrastructure & Quality Assurance....20
1. Local Guardrails: pre-commit & Ruff....20
2. AI-Assisted Review: CodeRabbit....21
3. Timeline & Implementation Schedule....21
3.1. Community Bonding Phase....21
3.2. First Coding Phase: Standardization & Infrastructure....22
3.3. Second Coding Phase: Autopilot & Discoverability....23
Who I Am....24
Why I am the Right Fit for this Project....24
Prior Contributions to Mesa....25
Availability....25
5. Expected Outcomes & Final Deliverables....25
6. Long-Term Vision & Post-GSoC Commitment....26

1. Project Overview & Motivation

As Mesa has evolved, the mesa-examples repository has suffered from the classic open-source "bit-rot" problem. Examples break silently as the core API changes, documentation quality varies wildly, and the current flat-folder structure makes it incredibly difficult for new users to discover relevant models.

The goal of this project is not to manually fix broken models. Instead, the goal is to architect a sustainable system that automates repository maintenance, lowers the barrier to entry for new contributors, and generates a searchable gallery UI.

By shifting mesa-examples from a fragile monolith to an isolated, automated matrix, we can ensure that the repository gracefully maintains itself as Mesa continues to grow.

1.1. Target Audience (User Personas)

To ensure the infrastructure actually solves the community's problems, this project is designed around three distinct user personas:

1. **The Student Modeler (The Contributor)**: They want to share their simulation without having to learn complex CI/CD infrastructure. They need clear, simple instructions and an environment where their PRs aren't blocked by someone else's outdated code.
2. **The Learner (The Consumer)**: They are browsing the gallery to find inspiration. They need a visual, easily filterable UI (by domain and complexity) rather than an alphabetical list of GitHub folders.
3. **The Core Maintainer**: They want to make editorial decisions, not chase down broken dependencies. They need automated triage bots and decentralized ownership to minimize their maintenance load.

1.2. Scope & Philosophy

This proposal strictly adheres to the maintainers' stated philosophy: "Working > Perfect."

Project Size: **Medium (Integral Approach)**. This proposal addresses the entire lifecycle of an example—from submission to testing, display, and eventual deprecation.

In Scope (What I will build):

- **Pillar 1 (DevEx & Standardization)**: Automated Developer Experience tooling to lower the barrier to entry, enforce documentation via YAML frontmatter, and mechanize the PR triage process using GitHub Actions.
- **Pillar 2 (Infrastructure & Matrix CI)**: Strict dependency isolation and Dynamic Matrix CI testing to prevent cross-model failures, including a Weekly Watchdog to proactively detect "bit-rot" as the Mesa core evolves.
- **Pillar 3 (Sustainable Maintenance)**: A decentralized autopilot system utilizing .github/CODEOWNERS and Graceful Degradation scripts to ensure the repository self-manages and drastically minimizes the core maintainers' administrative burden.
- **Pillar 4 (Discoverability & UI)**: A metadata-driven, serverless Static Site Generator (SSG) to transform the repository into a zero-cost, highly searchable UI gallery hosted natively on GitHub Pages.

Out of Scope (What I will avoid):

- **External Database/Server Hosting**: To keep maintenance near-zero, the gallery UI will rely entirely on Native GitHub Actions and Static Site Generation (GitHub Pages). I will not introduce external dependencies like PostgreSQL databases, AWS, or Heroku servers to host the gallery.
- **Headless UI Validation**: Visual UI rendering (e.g., Solara) is notoriously flaky in headless CI environments. CI will focus strictly on backend execution and dependency resolution, while visual validation will be enforced via the PR template screenshot requirement.

2. Technical Architecture

The core of this proposal relies on separating the concerns of the repository. Rather than treating mesa-examples as a single application, the architecture treats it as an automated gallery where each model operates in a sandboxed ecosystem.

2.1. Pillar 1: Automated Developer Experience (DevEx) & Standardization

The Problem: Intimidating Templates & Unwritten Rules Because mesa-examples was split from the main core repository, it inherited highly technical Pull Request templates designed for the core API engineers from mesa. This creates unnecessary friction for modelers and domain experts who simply want to share a working simulation. Furthermore, relying on core maintainers to manually explain documentation standards or check if an author included a screenshot is an unsustainable drain on bandwidth.

Community Validation & Prior Experience

This friction is not hypothetical; I have experienced it firsthand as a contributor. While working on the color_patches example, I identified and resolved a synchronization issue where the Readme file was entirely out of sync with the underlying code. I fixed and documented this discrepancy in PR #346 (**MERGED**). This misalignment likely occurred because the original developer missed or forgot to update the documentation after a code change—a direct symptom of lacking a standardized template that mechanically insists on a PR checklist.

The Solution: Mechanizing the Contribution Pipeline

To keep contributing accessible, we must lower the barrier to entry while mechanically enforcing quality standards before a human maintainer ever reviews the code. This is achieved through a three-part standardization system:

1. **The Repository Rulebook (CONTRIBUTING.md)**
2. **The "Mini-Paper" Specification (README_TEMPLATE.md)**
3. **The Contribution Guardrails (PR Templates & Automation)**

1. The Repository Rulebook (CONTRIBUTING.md)

A comprehensive guide will be located at the repository root. Rather than just a brief "how-to," this document will formally outline the end-to-end lifecycle of a model within the mesa-examples ecosystem. It will cover:

- **Local Environment Setup**: Best practices for testing models locally before submission.
- **Strict Folder Structure Rules**: Mandating localized, isolated requirements.txt files to prevent dependency sprawl.
- **The 3-Stage Review Process**: Defining expectations for *Author Self-Review* (using the PR checklist), *Peer Review* (behavioral testing by the community), and *Maintainer Approval* (the final editorial call).
- **Lifecycle Criteria**: Explicitly defining how a model moves through the status tiers (detailed below).

2. The "Mini-Paper" Specification (README_TEMPLATE.md)

To support academic and learner personas, the repository will provide a standardized README blueprint for all new models. Adopting the maintainers' recent architectural suggestion from the discussion forums, this template mandates YAML frontmatter. This allows contributors to copy a single file structure that acts as both human-readable documentation and machine-readable metadata for future UI workflows, since these two go hand-in-hand.

The Standardized YAML Frontmatter:

```yaml
---
title: Title 1
authors: [@Nandha-kumar-S]
domain: [domain 1, domain 2]
complexity: intermediate # basic, intermediate, or advanced
features: [feature 1, feature 2]
mesa_version_min: "3.0"
required_python_version: "3.12"
status: verified # verified, needs upgrade, deprecated or showcase
owner: [@Nandha-kumar-S]
last_tested: null
---

## Abstract
One-paragraph summary of what this model does and why it's interesting.

## Model Description
Agents, rules, space, parameters...

## How to Run
Exact commands to get it working.
```

- **Domain, Complexity & Features**: Restricts categorization to a recommended taxonomy to ensure the UI gallery remains cleanly searchable.

- **Required Python Version (Future-Proofing the Runner):** Contributors must explicitly state the Python version their model was built for (e.g., required_python_version: "3.12"). This is critical metadata that will directly feed into the CI matrix, ensuring the model is always tested in its native environment rather than a hardcoded global version.

- **Status (The Lifecycle Engine)**: The status field is the most critical component for "Sustainable Maintenance". Based on the maintainers' vision, this metadata field categorizes models into a tiered lifecycle. I arrived at these four tags after gaining insights from discussions with peer contributors and maintainers (see discussion)

Status	What it means
Verified	The model has been peer-reviewed, documentation is complete, and it is actively maintained
Needs upgrade	The model runs but it is not yet polished or fully documented
Deprecated	The model is no longer maintained or broken due to legacy dependencies
Showcase	High quality models selected by maintainers as exemplary

3. The Contribution Guardrails (PR Templates & Automation)

To prevent the exact synchronization issues I solved in PR #346, a simplified PR checklist and an automated GitHub Actions bot will guide the user through submission, serving as a preventative measure and offloading routine triage from the core team.

a. The Custom PR Template (PULL_REQUEST_TEMPLATE.md):

This template serves as a mandatory self-review, explicitly reminding contributors to ensure their README.md is updated and aligned with their code changes. I have implemented this in my GSoC learning space

```
## Welcome to the Mesa Examples Gallery!
Thank you for sharing your model! To keep our gallery running smoothly, please complete this quick checklist.

### The Checklist
- [ ] **Dependency Isolation:** I have included a `requirements.txt` inside my model's folder.
- [ ] **YAML Frontmatter:**  I have included a `README.md` based on the template, ensuring documentation is synced with the code.
- [ ] **Visual Proof:** I have attached a screenshot or GIF of the model running below.

### 📸 Screenshot / GIF
*(Drag and drop your image here!)*

### 📝 Brief Description
*(What does this model do? What Mesa features does it highlight?)*
```

b. The Automated Triage Bot (pr_greeter.yml):

I have incorporated a Triage Bot in my GSoC learning space - that utilizes actions/github-script with securely scoped pull-requests: write permissions. It automatically comments on new Pull Requests, ensuring contributors feel immediately welcomed while setting clear expectations:

```yaml
name: PR Greeter Bot

on:
  pull_request:
    types: [opened]

jobs:
  welcome-triage:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write # This gives the bot permission to comment
    steps:
      - name: Post Welcome Comment
        uses: actions/github-script@v7
        with:
          script: |
            const body = `
            👋 **Welcome to the \`mesa-examples\` gallery!** Thank you for your contribution, @${context.actor}!

            🤖 **Automated Checks are running...**
            Our CI pipeline is currently spinning up an isolated environment to test your model against the latest version of Mesa.

            While you wait, please ensure you have filled out the PR template checklist above (especially the screenshot!). A human maintainer will review your model soon. Keep up the great work! 🚀
            `;
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: body
            })
```

2.2. Pillar 2: Strict Isolation & Dynamic Matrix CI (The Infrastructure)

The Problem: The "Fragile Monolith" & Version Traps Currently, the repository relies on a global CI workflow (test_examples.yml) that triggers on any push and sequentially runs a global test_examples.py script. It attempts to install all dependencies for all models globally, including pulling the bleeding-edge Mesa development branch (mesa@main).

Community Validation & Prior Experience: As an active contributor, I experienced the flaw of this architecture firsthand in PR #343 (Traffic Flow Model). My model was built for the stable mesa-3.x release, but the global CI forcefully installed Mesa 4.0.dev0, causing immediate failures. To mitigate this, the maintainers recently had to introduce a mesa-3.x branch (Issue #386). While branching per major version is a helpful immediate patch, it is not a scalable long-term architecture; creating a new branch for every future Mesa release will severely fragment the repository.

Furthermore, attempting to consolidate all dependencies into one global requirements.txt with the "latest" versions simply shifts the burden. It creates version conflicts where older, perfectly

functional models fail the CI due to new PRs requiring newer libraries—a point I recently raised and validated with the community in recent architectural discussions (Discussion #417).

The Solution: Sandboxed Matrix Environments Rather than forcing all models to share a single environment or repository branch, we must enforce strict per-model dependency isolation and utilize a dynamic GitHub Actions Matrix. This solution comprises three blocks as follows:

1. **Dependency Isolation**
2. **Sparse Testing via Dynamic Matrix**
3. **The Weekly Watchdog & Automated Maintenance**

1. Dependency Isolation (The Foundation)

The core reason models break silently is the global installation script. We will enforce a strict rule that every single model directory MUST contain its own requirements.txt.

- **The Guardrail**: As established in Pillar 1, the automated PR Template explicitly requires contributors to verify they have isolated their dependencies.
- **The Benefit**: This guarantees that UI-heavy models requiring specific libraries (like Solara or NetworkX) and math-heavy models do not clash or fail due to global dependency resolution errors. If a 2023 model works perfectly on Mesa 2.1 and Pandas 1.5, it is allowed to live peacefully in its isolated environment.

2. Sparse Testing via Dynamic Matrix (The Gatekeeper)

I propose completely rewriting the current test_examples.yml workflow. Instead of one script testing everything simultaneously, the CI will utilize Dynamic Matrix Sparse Testing.

- **The Execution:** A detector job runs git diff --name-only origin/main...HEAD to output a JSON array of the specific directories modified by the contributor. The pipeline dynamically absorbs this array and spins up isolated Ubuntu containers *exclusively* for the modified models.
- **The Benefit**: If someone submits a broken model, it fails in isolation. The rest of the repository's tests remain green, eliminating the "Hostage PR" problem entirely.

Addressing the Python Version Trap (Dynamic Provisioning) A fatal flaw in many CI pipelines is hardcoding a single runner environment (e.g., python-version: "3.12"). As the Python language evolves, this creates a ticking time bomb.

- **The Consequence:** If a contributor submits a model next year utilizing new Python 3.13 syntax, or if a legacy model relies on an older version of scipy that drops 3.12 support, the CI will fail the PR. It fails not because the Mesa code is broken, but because the runner environment is obsolete. This "Dependency Hell" forces maintainers to constantly manually patch the CI workflow to juggle versions.
- **The Solution:** To make this architecture truly future-proof, my Dynamic Matrix CI will parse the required_python_version directly from the model's YAML frontmatter. The git diff detector script will extract this value and pass it into the GitHub Actions JSON array. This allows the Matrix to **dynamically provision the exact Ubuntu + Python environment** that specific model requires. For example, the CI can simultaneously spin up a Python 3.11 runner for an older 2023 model, and a Python 3.13 runner for a brand new PR, all within the same automated workflow.

Implementation Snapshot (Dynamic Matrix CI):

```yaml
name: Mesa Examples Matrix CI
on: [pull_request]

jobs:
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.set-matrix.outputs.matrix }}
    steps:
      - uses: actions/checkout@v4
      - name: Detect modified example folders
        id: set-matrix
        run: |
          # Outputs JSON array of strictly modified folders
          echo "matrix=[\"examples/traffic_flow\"]" >> $GITHUB_OUTPUT

  test-models:
    needs: detect-changes
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false # Crucial: Keeps healthy PR tests running even if one legacy model fails
      matrix:
        folder: ${{ fromJson(needs.detect-changes.outputs.matrix) }}
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python with Pip Caching
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip' # Prevents wasteful duplicate downloads across parallel runners
      - name: Isolated Install & Run
        run: |
          cd ${{ matrix.folder }}
          pip install -r requirements.txt
          pytest .
```

3. The Weekly Watchdog & Automated Maintenance (Bit-Rot Detection)

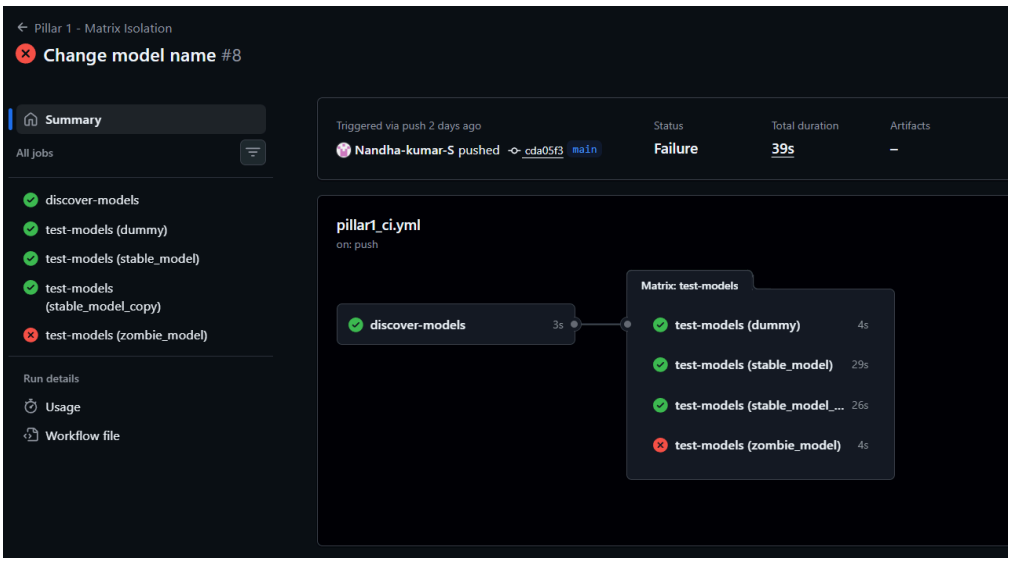
While PRs use git diff to test *only* changed files, we must proactively monitor untouched models for "bit-rot" as the Mesa core API evolves. A separate GitHub Action (schedule: cron '0 0 * * 0') will run every Sunday to automatically test models against the bleeding-edge mesa main branch.

The Automated Watchdog Flow:

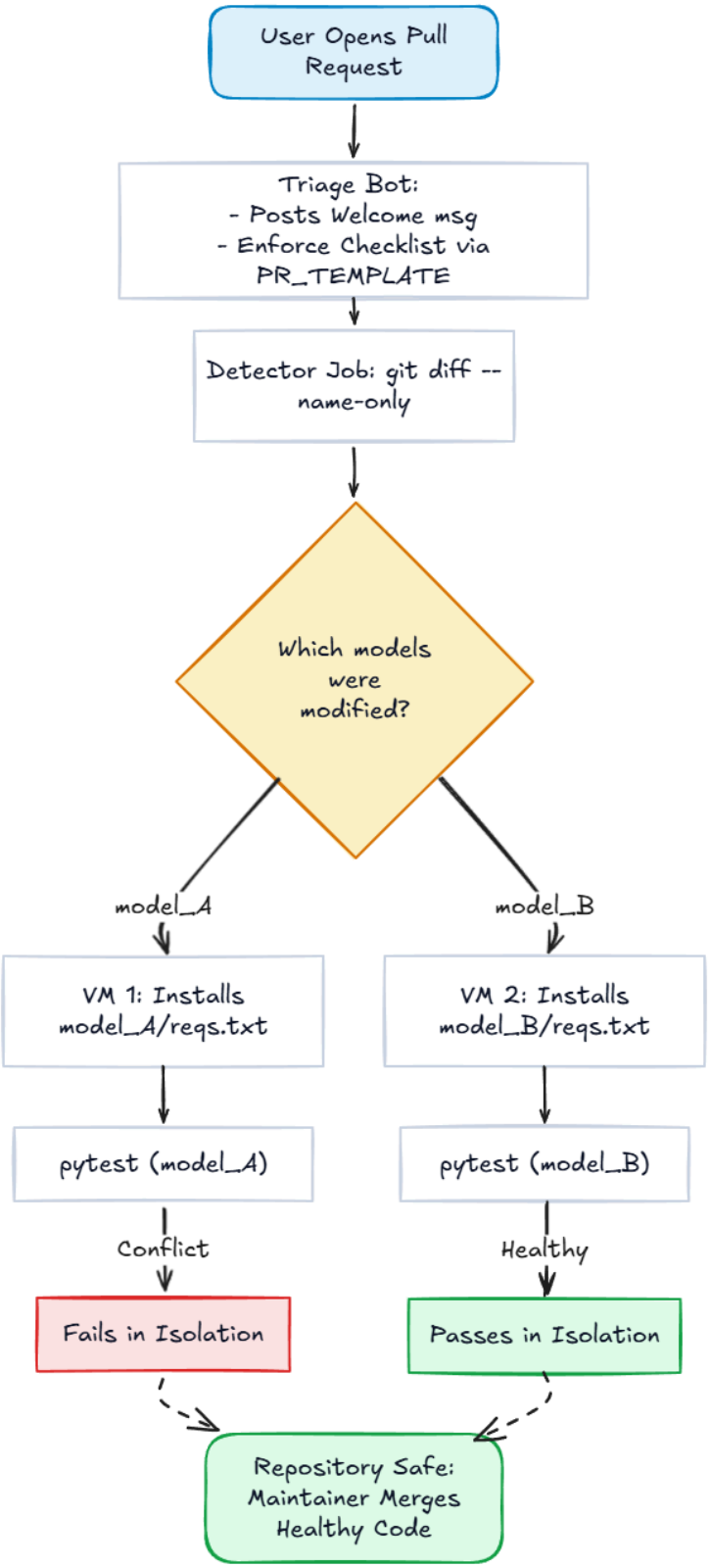
1. **Test Execution:** The matrix crawls all model folders and runs their isolated tests against Mesa main.
2. **Compatibility Matrix**: The workflow automatically outputs a "Compatibility Matrix" (a Markdown table in the repository) showing users exactly which models work with Mesa v3.0 vs. v4.0.
3. **Graceful Degradation (Metadata Updates):** If a model passes, a Python script automatically updates the last_tested YAML frontmatter variable with the current date.
4. **Automated Issue Routing**: If a status: verified model breaks due to a new Mesa update, the CI will automatically open a GitHub Issue, paste the error log, and ping the original author via the CODEOWNERS registry (detailed in Pillar 3). If it remains broken beyond a grace period, the script automatically changes its YAML status to needs-upgrade or deprecated, ensuring the UI gallery always reflects reality.

Proof of Concept: To ensure the viability of this architecture, I built and tested a fully functional prototype of this Matrix CI in my sandbox repository. You can view my live GitHub Actions execution.

The screenshot below demonstrates the CI successfully isolating a healthy models (stable_model, dummy, stable_model_copy), which passes, while simultaneously catching a broken model (zombie_model), which fails independently without crashing the entire workflow.



Flowchart for Pillars 1 & 2 Implementation:



2.3. Pillar 3: Sustainable Maintenance & Decentralized Ownership (The Autopilot)

The Problem: "Orphaned Code" & Maintainer Burnout The open-source contribution lifecycle has a predictable pattern: a student builds a brilliant model, submits a PR, gets it merged, and eventually graduates and moves on. Years later, when Mesa releases a major update (e.g., v4.0), that model breaks. If the repository lacks an automated ownership protocol, the core maintainers are stuck with a lose-lose choice: spend their weekends fixing someone else's legacy code, or manually delete the broken model.

Community Validation & Maintainer Goals: In the recent community roadmap discussions, the maintainers explicitly stated their primary goal: *"Minimize maintainer load. Maintainers should make editorial decisions, not chase down every broken example."* Some proposed solutions in the community involve routing CI failure alerts directly to the core team. This fundamentally violates the maintainers' goal. We must shift the burden of maintenance away from the core team and distribute it to the original authors and automated bots.

The Solution: A Decentralized Autopilot System To ensure the repository maintains itself without administrative overhead, I will implement a decentralized ownership architecture paired with proactive dependency bots and a "Graceful Degradation" protocol. This solution is structured into three main blocks as follows:

1. **Decentralized Ownership**
2. **Proactive Dependency Management**
3. **Graceful Degradation**

1. Decentralized Ownership (CODEOWNERS)

Instead of treating the core maintainers as the default owners of all examples, we will utilize GitHub's native .github/CODEOWNERS feature. This maps specific example folders strictly to the GitHub usernames of their original authors.

- **Automated Routing:** When the Weekly Watchdog (from Pillar 2) detects a failure in an isolated model, it automatically opens a GitHub Issue. Because of the CODEOWNERS file, GitHub bypasses the core maintainers entirely and automatically assigns the issue to the original author.
- **Zero Admin Overhead:** To prevent maintainers from having to manually edit the CODEOWNERS file every time a new PR is merged, I will implement a lightweight GitHub Action that automatically appends the new contributor's handle to the file upon merge.

Implementation Snapshot The Ownership Map (.github/CODEOWNERS):

```
# ============================================================
# MESA EXAMPLES: CODE OWNERSHIP MAP
# This file automatically requests reviews and assigns issues
# to the original authors of specific models.
# ============================================================

# 1. Core Infrastructure (Owned by Maintainers)
/.github/                   @maintainers
/build_gallery.py           @maintainers

# 2. Community Models (Owned by Original Authors)
/models/stable_model/     @Nandha-kumar-S
/models/zombie_model/       @Nandha-kumar-S

# (In the final GSoC project, this section will be dynamically
# updated by a bot whenever a new model is merged!)
```

2. Proactive Dependency Management (Dependabot)

Instead of waiting for a user to complain that a model will not install, the repository will utilize Dependabot to proactively monitor the isolated environments.

- A .github/dependabot.yml configuration will be deployed to scan the localized requirements.txt files on a monthly schedule.
- If a minor update is released for a package, Dependabot automatically opens a PR, runs the Dynamic Matrix CI (Pillar 2) to ensure the update doesn't break the model, and requests a review from the designated CODEOWNER.

Implementation Snapshot Proactive Updates (.github/dependabot.yml):

```yaml
version: 2
updates:
  # 1. Maintain the GitHub Actions infrastructure
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
    labels:
      - "infrastructure"

  # 2. Maintain the Python dependencies for the Stable Model
  - package-ecosystem: "pip"
    directory: "/models/stable_model"
    schedule:
      interval: "monthly"
    labels:
      - "dependencies"
      - "stable_model"
```

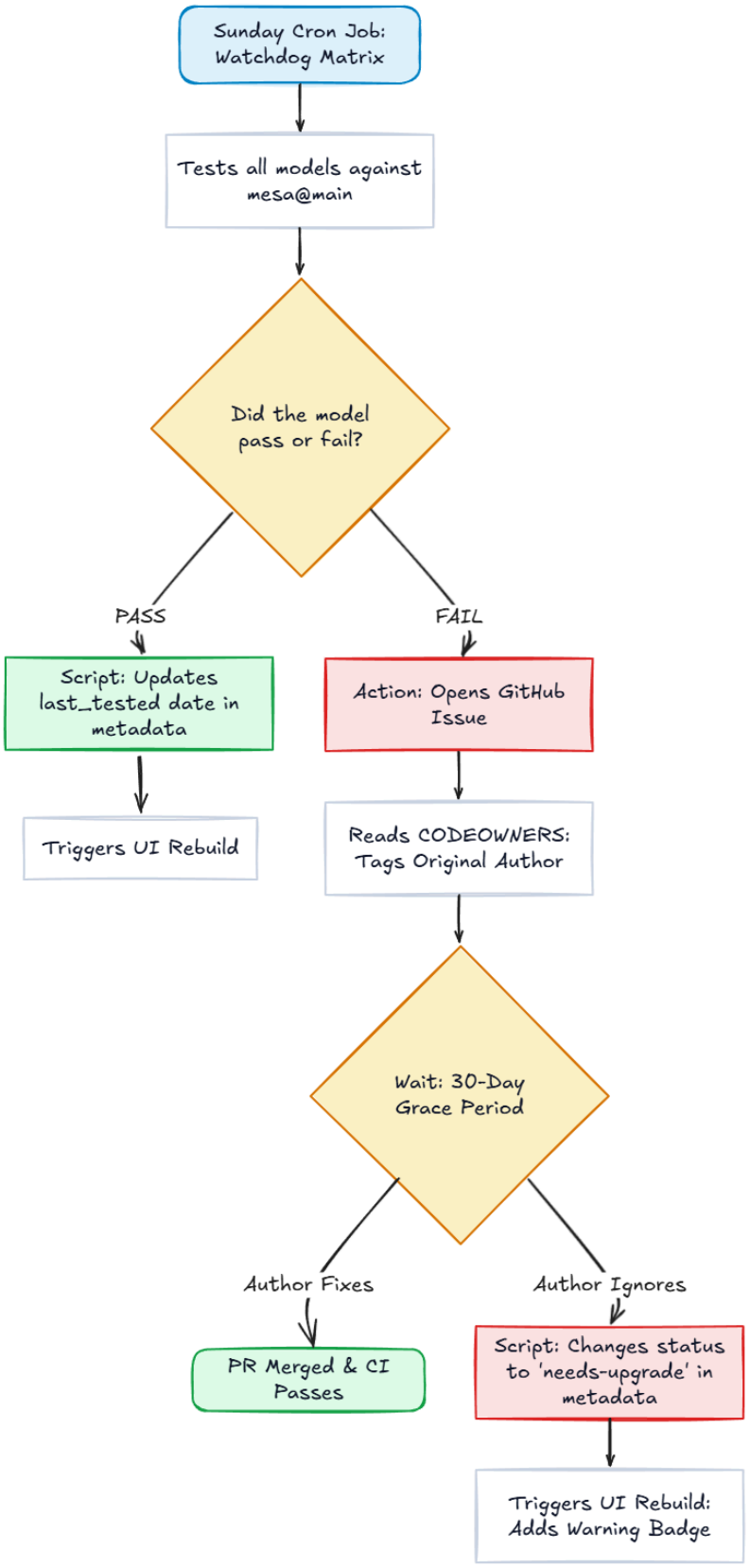
3. Graceful Degradation (The Lifecycle Loop)

If an original author ignores the automated GitHub ping and abandons their model, the system must handle it gracefully.

- If an automated issue regarding a broken model remains open and unaddressed for a specific grace period (e.g., 30 days), an automated script will safely update the model's README.md YAML frontmatter, changing its status from verified to needs-upgrade or deprecated.
- As a result, the gallery UI updates automatically to warn users, and the broken model is cleanly categorized without the core maintainers lifting a finger.

Proof of Concept: To prove that this decentralized architecture successfully protects the core repository, I configured these two autopilot systems in my sandbox environment. The CODEOWNERS (link) mapping successfully isolates folder responsibility, while Dependabot (link) actively monitors the pip ecosystems of the individual model folders without creating global dependency conflicts.

Flowchart for Pillar 3 Implementation:



2.4. Pillar 4: Metadata-Driven Discoverability (The Gallery UI)

The Problem: "Folder Fatigue" & The Hosting Dilemma Currently, the mesa-examples repository is essentially a flat alphabetical list of directories. If a researcher visits the repository looking for an "intermediate-level economics model," they are forced to blindly click through all the model folders hoping to find what they need. This structure buries high-quality models and is hostile to the "Learner" persona.

A modern framework requires a modern, searchable showcase. However, building a traditional web application with a backend database (like PostgreSQL or MongoDB) introduces unacceptable server costs and massive security/maintenance overhead for an open-source project.

The Solution: A Serverless Static Site Generator (SSG) To solve discoverability without introducing server maintenance, I will architect a custom, lightweight Static Site Generator (SSG) natively integrated into the repository. The philosophy is simple: treat the repository itself as the database, separate the data (YAML frontmatter) from the presentation (HTML/CSS), and leverage GitHub Pages for zero-cost hosting.

The Implementation Flow (The Data Pipeline)

The gallery generation operates as a three-step automated data pipeline requiring zero human intervention:

Step 1: The Repository as a Database (Metadata Crawler) I will develop a Python utility script (build_gallery.py) that acts as a repository crawler.

- Triggered by a GitHub Action, this script utilizes the pyyaml and python-frontmatter libraries to iterate through every model folder.
- It extracts the standardized YAML frontmatter from each model's README.md (Title, Domain, Complexity, Status, etc.).
- It compiles all of this extracted data into a single, structured registry.json file.

Step 2: The Templating Engine (Jinja2 & Vanilla JS) Once the registry.json is built, the visual layer is generated.

- Using the Jinja2 Python templating engine, the script injects the JSON data into a master index.html template.
- To keep the UI lightning-fast and dependency-free, the frontend will utilize a lightweight CSS framework (e.g., Tailwind CSS via CDN) paired with Vanilla JavaScript.

- When a user visits the site, their browser downloads the registry.json payload, powering client-side, real-time filtering (e.g., clicking a "Social Dynamics" or "Verified" tag instantly filters the visible model cards without ever querying a backend server).

Step 3: Zero-Cost Deployment & Hosting (GitHub Pages) The gallery requires zero external hosting environments (no AWS, no Heroku).

- A dedicated deploy_gallery.yml GitHub Action will be configured to automatically build the site and push the output to the repository's gh-pages branch.
- GitHub Pages natively detects this branch and serves the static website globally at the standard URL (e.g., https://mesa.github.io/mesa-examples/).
- The Autopilot Connection: This deployment action is triggered on: push to the main branch, but crucially, it is also triggered by the Weekly Watchdog (from Pillar 2/3). If the **Watchdog** degrades a model's status to needs-upgrade in the YAML file, the gallery immediately rebuilds. The broken model's UI card instantly updates with a visual **"Needs Upgrade"** badge.

User Access & Navigation

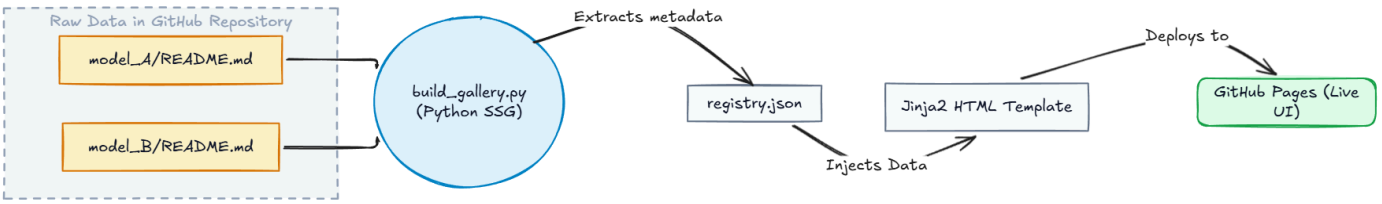
To ensure maximum discoverability, the generated GitHub Pages link will be embedded in three strategic locations: the repository's native "About" sidebar, prominently at the top of the root README.md, and integrated into the main Mesa library's official documentation site.

UI Gallery Features:

- **Visual Model Cards**: Each example is rendered as a clean card displaying its title, abstract, complexity, and dynamically color-coded status badges.
- **Faceted Search & Filtering**: Users can filter the repository by Domain (e.g., Economics, Biology), Complexity (Basic, Advanced), or Features (e.g., Solara UI, NetworkX).
- **Direct Repository Linking**: Clicking a card redirects the user directly to the model's source code on GitHub.

Proof of Concept: To validate this serverless SSG approach, I developed a prototype in my sandbox repository. I wrote the foundational build_gallery.py script (link), which successfully parses dummy metadata and generates a responsive HTML grid layout dynamically. **Screenshot attached below.**

Flowchart for Pillar 4 Implementation:



2.4.1. Anatomy of the Proposed UI (Feature Breakdown)

The proposed Static Site Gallery is designed specifically to serve the "Learner" and "Researcher" personas, transforming a flat repository into a highly discoverable, frictionless web experience. Every visual element on the page is dynamically generated from the backend YAML and CI metadata workflows outlined in previous pillars.

1. Client-Side Faceted Filtering (The Sidebar) Because the Python SSG compiles all repository data into a single registry.json payload, the UI can perform lightning-fast, client-side filtering without querying a backend database.

- Users can instantly filter the repository by **Domain** (e.g., Economics, Social Dynamics), **Complexity** (Beginner, Advanced), or **Mesa Features** (e.g., Solara UI). This data is pulled strictly from the standardized README.md YAML frontmatter (Pillar 1).

2. Dynamic "Health" Badges (Automated by Pillar 3) The UI serves as a real-time dashboard for repository health.

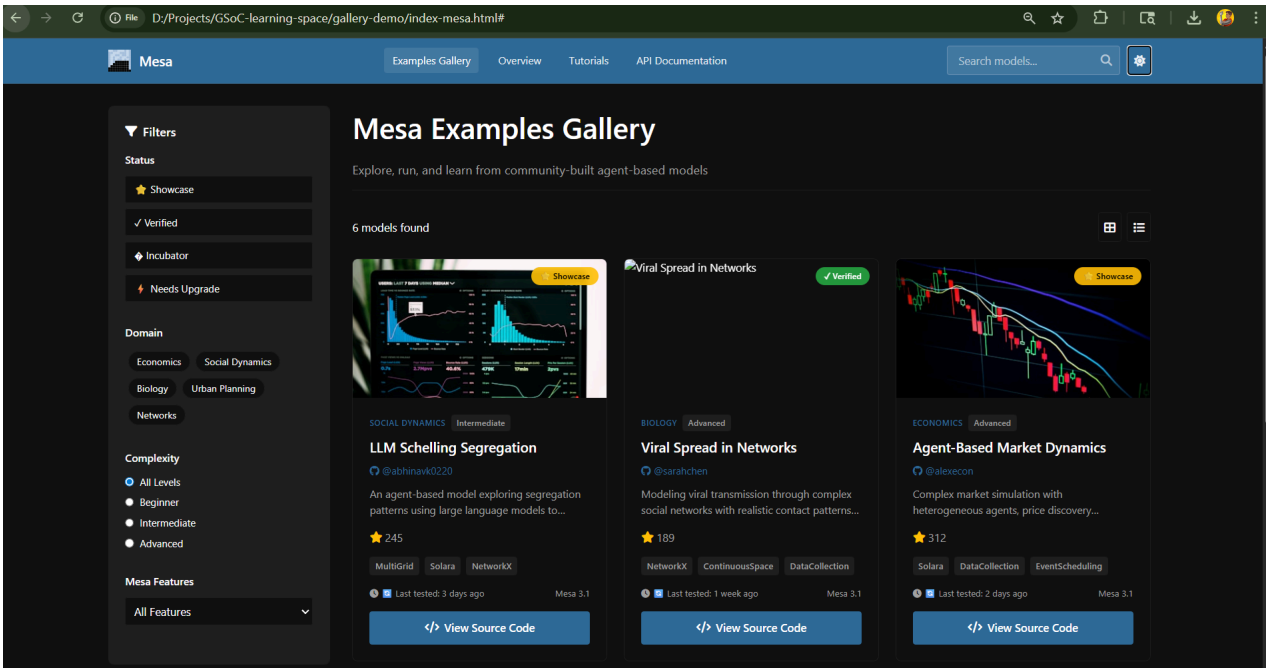
- Every model card features a prominent status badge (e.g., a green Verified badge or a red Needs Upgrade badge).
- Crucially, these are not manually updated by maintainers. If the Weekly Watchdog (Pillar 2) detects a failure and the 30-day grace period expires (Pillar 3), the SSG automatically rebuilds the site to swap the badge to Needs Upgrade, warning users before they even click the model.

3. The Standardized Model Card (The Grid) To eliminate "Folder Fatigue," the models are presented in a clean, visual grid. Each card dynamically displays:

- Visual Proof: The screenshot or GIF mandated by the PR Template (Pillar 1).
- Attribution: The original author's handle, linked directly to their GitHub profile via the .github/CODEOWNERS mapping.

- Abstract & Metrics: A brief summary and the "Last Tested" date, proving the model is compatible with the latest Mesa releases.
- Frictionless Access: A "View Source Code" button that routes the user directly to that specific isolated folder within the mesa-examples repository.

The screenshot below demonstrates the proposed UI, proving that a high-quality, searchable interface can be built entirely through Python scripting and static HTML, requiring absolutely no database infrastructure.



Supporting Infrastructure & Quality Assurance

My proposed architecture is designed to integrate seamlessly with—and elevate—the existing quality-assurance tooling currently utilized within the mesa-examples repository.

1. Local Guardrails: pre-commit & Ruff

The repository currently employs pre-commit hooks to enforce Pythonic style via Ruff.

- **Integration with Pillar 1:** My standardization of the README_TEMPLATE.md and requirements.txt will leverage these hooks to ensure that new contributions are not only functionally isolated but also visually consistent.
- **Benefit**: This prevents "lint-war" discussions in Pull Requests, ensuring that the **Gallery UI (Pillar 4)** displays clean, consistently formatted metadata and code snippets.

2. AI-Assisted Review: CodeRabbit

Mesa utilizes CodeRabbit to provide automated, AI-driven summaries and code reviews on every Pull Request.

- Integration with Pillar 3: While CodeRabbit provides a high-level "first pass" on code logic, my **Automated Watchdog** and **CODEOWNERS routing system (Pillar 3)** handle the technical task of long-term API compatibility.
- **Benefit**: Together, these tools create a "Force Multiplier" for maintainers. CodeRabbit handles the initial review feedback, while my system handles the long-term maintenance lifecycle, collectively reducing human intervention to a bare minimum.

3. Timeline & Implementation Schedule

3.1. Community Bonding Phase

During this phase, I want to focus on discussing the final architectural decisions with the mentors and the broader Mesa community to ensure alignment before writing production code. Key objectives include:

- **Finalizing the YAML Taxonomy**: Collaborating with maintainers to lock in the official "Recommended Domains" and "Complexity" tags for the README_TEMPLATE.md.
- **Sandbox Verification**: Reviewing the sandbox deployments of the GitHub Actions (Triage Bot and Matrix CI) with mentors to ensure the security scopes (pull-requests: write) meet the organization's repository standards.
- **Legacy Model Audit**: Identifying the first batch of 5–10 high-priority legacy models that will serve as the initial test cases for the strict dependency isolation migration.

3.2. First Coding Phase: Standardization & Infrastructure

This phase focuses on building the foundational "plumbing" of the repository. By the end of this phase, the repository will automatically enforce contribution standards and run isolated tests.

Week	Focus Area	Implementation Tasks
Week 1	DevEx (Pillar 1)	Draft CONTRIBUTING.md, create README_TEMPLATE.md with YAML frontmatter, and configure pre-commit hooks for modified files.
Week 2	Automation (Pillar 1)	Implement the Custom PR Checklist and deploy the PR Greeter Bot via actions/github-script.
Week 3	Initial Migration	Convert the first 5–10 legacy models to the new architecture (isolating requirements.txt and adding YAML metadata).
Week 4	Matrix CI (Pillar 2)	Develop the git diff --name-only detector script to identify modified example directories for sparse testing.
Week 5	Matrix CI (Pillar 2)	Build the test_examples.yml workflow, configuring parallel Ubuntu testing environments.
Week 6	Stress Testing	Intentionally introduce dependency conflicts in a test branch to prove the Matrix isolates failures without crashing the global workflow.

➔ **Midterm Evaluation:** Deliverables include a fully standardized contribution pipeline, the deprecation of the fragile global CI script, and a functional Dynamic Matrix CI testing isolated environments.

3.3. Second Coding Phase: Autopilot & Discoverability

With the testing infrastructure secure, this phase focuses on automating long-term maintenance and building the external-facing UI.

Week	Focus Area	Implementation Tasks
Week 7	Watchdog (Pillar 2)	Develop the Weekly Watchdog cron job to test models against mesa@main and auto-update the last_tested YAML date.
Week 8	Ownership (Pillar 3)	Deploy .github/CODEOWNERS and write the automated GitHub Action to route Watchdog failure issues directly to original authors.
Week 9	Degradation (Pillar 3)	Implement the Graceful Degradation script to automatically update YAML status to needs-upgrade if an issue passes the 30-day grace period.
Week 10	Gallery UI (Pillar 4)	Develop build_gallery.py (Python SSG) to extract YAML metadata into registry.json. Design the Jinja2 HTML grid layout.
Week 11	Gallery UI (Pillar 4)	Write Vanilla JavaScript for real-time faceted filtering (Domain, Status) and configure continuous deployment to GitHub Pages.
Week 12	The Great Migration	Bulk-update remaining legacy models to populate the new UI Gallery. Finalize all user and maintainer documentation.

➔ **Final Evaluation:** Deliverables include a live, zero-cost UI Gallery hosted on GitHub Pages, and a completely self-maintaining repository ecosystem that flags broken code without maintainer intervention.

4. About Me & Background

Name: Nandhakumar

GitHub: https://github.com/Nandha-kumar-S

LinkedIn: https://www.linkedin.com/in/nandhakumar-s-12a415212/

Medium: https://medium.com/@suriyank532002

Email: suriyank532002@gmail.com

Current Role: Research Engineer at Saama Technologies

Education: B.Tech Information Technology

Who I Am

I am a Research engineer skilled primarily specialised in python development and open-source contributor with a deep focus on data standardization, CI/CD infrastructure, and scalable system architecture. I thrive at the intersection of backend automation and data structuring, building systems that convert messy, unstructured processes into clean, automated pipelines. My technical stack is heavily focused on **Python, GitHub Actions (CI/CD), Data Parsing (JSON/YAML), and modern workflow automation.**

Why I am the Right Fit for this Project

This proposal requires an engineer who understands how to build reliable infrastructure (Pillars 1 & 2) and how to cleanly extract and present data (Pillars 3 & 4). My recent experiences map perfectly to these exact challenges:

1. **DevOps & Infrastructure (The CI Matrix)**: In my current role as a Research Engineer, I gained hands-on experience designing and maintaining **GitHub Action workflows**. I understand the nuances of CI/CD pipelines, dependency caching, and automated testing, giving me the exact skillset required to build Mesa's new Dynamic Matrix CI.

2. **Data Standardization (The YAML to JSON Pipeline)**: I recently participated in the CDISC 25 Hackathon, where I showcased a solution for digitizing complex clinical protocol PDFs into a highly standardized structured format (USDM). Additionally, I am currently developing an **open-source Python tool** called pdfmind that semantically

extracts data from PDFs and outputs it as structured JSON.

3. **Execution Under Pressure**: I recently won the award for Best Technical Implementation at the Saama Hackathon | AI Catalyst 25 for protocol-library project. I am highly accustomed to delivering robust, production-ready code under tight deadlines and complex constraints.
4. **Academic Rigor:** I am currently co-authoring a research paper building a benchmark dataset for LLMs focused on applied science MCQs. I understand the rigorous needs of researchers and academics—the primary target audience for the Mesa framework.

Prior Contributions to Mesa

I am already an active contributor to the mesa-examples ecosystem and am deeply familiar with the current codebase and the maintainers' workflow. My recent merged contributions include:

- [PR #343] Traffic Flow: In this pull request, I have created the traffic_flow example which includes a CarAgent that moves based on cell availability and a TrafficFlow model class.
- [PR #346] Color Patches **(merged)**: In this pull request, I have updated the color_patches example to support Mesa 3.x SolaraViz, refactored the model to use a direct grid attribute instead of a legacy property, and modernized the README with the correct Solara run commands .
- Community Engagement: Beyond committing code, I actively participate in the repository's open discussions regarding architecture and best practices. I strongly believe in the open-source ethos of "building in public" and ensuring my engineering decisions align perfectly with the core maintainers' long-term vision.

Availability

I am fully committed to the GSoC 2026 timeline. I will be dedicating 30 hours per week to this project during the coding period, ensuring that every milestone outlined in my timeline is met with high-quality, fully tested code.

5. Expected Outcomes & Final Deliverables

By the end of the Google Summer of Code 2026 coding period, the mesa-examples repository will be transformed from a flat, manually managed folder structure into a highly automated, self-healing ecosystem.

The Mesa core maintainers will receive the following fully tested and documented deliverables:

- **Standardized Contribution Pipeline:** A new CONTRIBUTING.md, a mandatory PULL_REQUEST_TEMPLATE.md, and an automated GitHub Action Triage Bot to enforce YAML frontmatter and visual proofs on all new models.
- **The Dynamic Matrix CI:** A custom git diff sparse-testing workflow (test_examples.yml) that dynamically provisions isolated parallel environments, ensuring dependency conflicts in one model never crash the global repository.
- **The 'Autopilot' Maintenance Loop:** A Weekly Watchdog cron job that automatically tests all models against mesa@main, complete with a decentralized .github/CODEOWNERS routing system and a Graceful Degradation script that flags broken models without requiring maintainer intervention.
- **The Serverless UI Gallery:** A custom Python Static Site Generator (build_gallery.py) that compiles repository YAML into a registry.json payload, powering a beautiful, lightning-fast, real-time searchable UI hosted entirely for free on GitHub Pages.
- **The Great Migration**: A baseline of high-priority legacy models fully converted to the new architecture, serving as the live data for the UI Gallery and the template for future community contributions.
- **Maintainer Documentation**: A comprehensive internal guide detailing how the new CI workflows and SSG scripts operate, ensuring the Mesa team can easily manage or tweak the automation logic in the future.

6. Long-Term Vision & Post-GSoC Commitment

I do not view Google Summer of Code as a simple three-month summer job; I view it as an onboarding period to become a long-term core maintainer of the mesa-examples ecosystem.

Even after the official GSoC period concludes, I am highly determined to stay involved with the Mesa community. My long-term vision is split into two main areas of focus:

1. Infrastructure Maintenance & Contributor Support Infrastructure is never truly "finished." As GitHub Actions evolve and new Python versions are released, the Dynamic Matrix CI and the Watchdog scripts will require occasional maintenance. Post-GSoC, I intend to:

- **Actively monitor** the automated Watchdog issues and help authors fix their broken models.
- **Serve as a reviewer** for incoming PRs, specifically helping new contributors understand the CONTRIBUTING.md standards and the YAML frontmatter requirements.
- **Continue refining the UI Gallery** (Pillar 4) by adding new filtering features or integrating it deeper into the main Mesa documentation.

2. Evangelism & Content Creation Because I am an active technical writer on Medium, I want to leverage my platform to attract new users and contributors to the Mesa framework. Following the completion of GSoC, I plan to publish a series of articles:

- **The Architecture Deep-Dive**: A technical blog post detailing how we built a zero-cost, serverless UI Gallery and CI testing matrix for Mesa, which will attract DevOps-minded engineers to the project.
- **ABM in Action**: A detailed tutorial on Agent-Based Modeling using my traffic_flow model. I plan to build additional enhancements into this model and use it as a showcase piece on Medium to demonstrate the power and flexibility of the Mesa core engine to the broader Python data science community.

Given my professional commitments and alignment with my ongoing research, I can comfortably reserve time each week to continue contributing to Mesa. I am excited to help this community grow and ensure that mesa-examples remains the premier destination for learning agent-based modeling in Python.

7. References & Resources

- GitHub Actions Matrix Strategy: docs.github.com/actions/using-a-matrix
- GitHub Triage Bot: github.com/actions/github-script
- Decentralized Code Ownership: docs.github.com/about-code-owners
- Pre-commit Hooks: pre-commit.com
- Ruff (The Python Linter): docs.astral.sh/ruff
- Jinja2 (HTML Templating Engine): jinja.palletsprojects.com
- Hugging Face Spaces: huggingface.co/spaces
- Vercel Templates Gallery: vercel.com/templates